

Superjoin Fact Knowledge Layer — Review of Your Plan

v2 — revised after a second-opinion cross-review, with one new dataset finding

Reviewed for Pratham Kashyap

September 7, 2026 · Deadline: September 9, 2026, 11:00 AM

TL;DR

The plan (PDF parse → LLM extract → normalize → SQLite → candidate matching → LLM reasoner → FastAPI/HTML) is the right shape and the schema instinct (subject/predicate/value/unit/period/scope) is sound. Two things need to change before you start coding:

1. **The starter dataset is bigger and messier than the plan assumes** — 6 PDFs, ~511 pages, ~20 MB, with the *same company reporting three different “revenue” numbers for the same year*. That changes both your performance budget and your schema.
2. **The sequencing puts the highest-risk step (LLM extraction + reasoning quality) in the middle of day two**. If that step underperforms, you find out with ~12 hours left instead of ~40. It should be the first thing you validate, today, on one page, before any scaffolding.

Everything below is ranked by how much it changes your outcome, not by how interesting it is to build.

0. Round 2 — verdict on the counter-review

You ran this past a second AI. It agreed with most of v1 and pushed back on one thing. Here’s my honest read of its pushback, plus one new finding of my own that neither of us had checked.

Their point	My verdict	Why
Replace “sampling” with deterministic candidate-page prioritization (keyword/structure signals, not arbitrary page skipping)	Agree — adopted	Better generalization story, and it’s honestly not much more work than sampling. See 2.1 (revised).
Broaden the spike to 3 micro-tests (extraction, GDP reconciliation, revenue-scope reconciliation) before building	Agree — adopted, added a 4th test	Cheap to do, catches schema problems before they’re load-bearing. See Section 4.

Their point	My verdict	Why
Formalize a FactComparison/comparison_dimensions object alongside Fact	Agree — adopted	Makes the reasoning trace an actual data structure instead of a string, which is easy to render in the UI and easy to defend in the README. See 2.5.
Defer embeddings until you actually hit a matching failure, not upfront	Agree — adopted	Correct YAGNI call given the clock. Canonical string normalization will cover most of the starter dataset.
Reframe Case 4 as “naive extractor fails → we found it → schema fix → correctly reconciled” rather than a static broken example	Agree, with one condition	Stronger narrative <i>only if</i> the naive failure is actually captured (a saved output, a screenshot, a failing test) — not just described after the fact. A fixed bug you can’t show evidence of reads the same as a bug you never had. See Section 5.
Hierarchical classifier: deterministic scope/period diff auto-routes to RECONCILABLE before the LLM ever sees it	Partially agree	The cost/determinism win is real, but hard-coding “different scope → automatically RECONCILABLE” is itself a small hard-coded rule, and it can be wrong (a scope difference can also be a genuine extraction bug, not a real reconciliation). Better: feed the deterministic diff into the LLM prompt as a strong hint (“these facts differ in scope: standalone vs. consolidated — classify accordingly”), and let the LLM make the final call. You keep almost all the cost savings without hand-authoring a rule that overrides the model’s judgment.

What I found that neither review caught: both v1 and the counter-review only looked at numeric facts (GDP %, revenue). The assignment’s own example of a reconcilable case is explicitly *non-numeric* — “a director may appear active in one document and resigned in a later one” — and that exact scenario is sitting in your dataset. See new section 2.6. This matters because if your extraction pipeline only knows how to pull out numbers-with-units, you’ll never produce this case, and it’s the one the assignment itself uses as its example.

1. What's already right — keep this

- **Fact schema instinct.** Splitting value from unit/period/scope instead of one blob is exactly what the assignment is testing for — see Section 2, it's not optional, it's load-bearing.
 - **Two-stage reasoning** (candidate grouping, then LLM classifies the pair) instead of all-pairs comparison. Correct call, keeps cost bounded.
 - **Deliberately including a failure case** rather than hiding one. The brief asks for this explicitly, and most candidates will skip it.
 - **Refusing React/graph-DB/RAG-framework scope creep.** The brief says this directly, and the plan already resists it.
-

2. What the real dataset changes

I unzipped `starter-datasets.zip` and looked at the actual PDFs instead of just the assignment description. This matters because your plan's examples (the ₹5,000M vs ₹6,000M reconciliation, the "8,460 vs 8,460 million" failure) are invented placeholders — and the real files already contain better, free versions of all four required cases.

2.1 It's not one folder — it's two independent datasets

`starter-datasets/` splits into `delhivery/` (3 corporate documents) and `india-macroeconomy/` (3 institutional reports). **6 PDFs, 511 pages total, ~20 MB.** Two of the Delhivery PDFs and two of the macro PDFs are ~100 pages each. That is not a "process everything through the LLM page by page" budget inside a two-day window — it's a real cost/latency constraint, not a hypothetical "brownie point" one.

Confidence: high — this is a direct file inspection, not an inference.

Action (revised in v2): don't arbitrarily sample pages ("process every 5th page" or "first N pages") — that degrades gracefully on the starter set but has no principled reason to work on an unseen PDF. Instead, run a cheap, deterministic first pass over every page *before* any LLM call, and only send candidate pages to the LLM:

all pages

- cheap local scoring (no LLM):
 - numeric density, currency symbols, %, tables,
 - dates/FY-patterns, headings, and a generic
 - keyword set (revenue, growth, profit, appointed,
 - resigned, estimate, consolidated, standalone, ...)
- ranked candidate pages
- LLM extraction on the candidates

This is still a form of prioritization, not full coverage — but it's a *method*, not a cutoff, so it runs unchanged on a document you've never seen. Log which pages were selected and why (even just the score) and put that logic in the "Limitations" section of the README. That combination — a generic, inspectable method plus honest disclosure — is what separates "reasonable engineering trade-off" from "hard-coded to the starter set" in a grader's eyes.

One caution on the keyword list itself: don't make it purely numeric/financial (revenue, GDP, %). See 2.5 — a page with a director resignation or an address change has almost no numeric density and would score near zero on a numbers-only heuristic, but it's exactly the kind of fact the assignment cares about.

2.2 A real corroboration case already exists — use it, don't invent one

Searching the three macro PDFs for GDP growth:

- **RBI Annual Report 2024-25:** growth “moderated to 6.5 per cent in 2024-25.”
- **IMF Article IV 2025:** “India’s real GDP grew by 6.5 percent in FY2024/25.”

Same metric, same period, two independent institutions, different phrasing. That’s Case 1, for free, with zero synthetic data needed.

2.3 A real “apparent contradiction” case already exists — and it’s a better story than a time-period one

- **Economic Survey 2024-25:** “real GDP is estimated to grow by 6.4 per cent in FY25” — explicitly sourced to the **First Advance Estimate** (an early, partial-data release).
- **RBI Annual Report** and **IMF Article IV:** both say **6.5 per cent** for the same year — based on **later, revised estimates**.

6.4% vs 6.5% looks like noise, but the real reconciling variable isn’t time *period* — it’s **data vintage / estimate type** (first advance estimate vs. provisional/revised estimate). That’s a subtler and more convincing demonstration of “grounded, compared, and explained” than an invented FY2023-vs-FY2024 example, because a grader can independently verify it from the source documents you were given.

This directly means your schema needs one more field — see 2.6.

2.4 A real extraction-failure trap already exists — this should be your Case 4

Inside the Delhivery Annual Report FY24 *alone*:

- Revenue from operations, **standalone** basis, FY24: INR 74,540.82 million.
- Revenue from operations, **consolidated** basis, FY24: INR 81,415.38 million.

And in the Q4 FY24 earnings presentation, a third number: “**revenue from services**”, which explicitly *excludes* revenue from traded goods.

Three different “revenue” figures, same company, same year, same-ish source family. A naive extractor that only captures value + unit + period (no scope/basis) will either (a) flag standalone-vs-consolidated as a contradiction, or (b) silently overwrite one with the other. Either outcome is a genuine, reproducible failure mode you can show honestly — “the extractor initially conflated these three revenue definitions; here’s how we detect and flag it” — instead of manufacturing a fake unit-loss bug.

Confidence: high. This is the single most useful thing I found — it’s a real bug your extractor *will* hit if you don’t design for it, not a contrived demo.

2.5 A real non-numeric reconciliation case — this matches the assignment’s own example almost exactly

The brief’s example of a reconcilable “apparent contradiction” isn’t a number at all: “*a director may appear active in one document and resigned in a later one.*” I checked, and your dataset has this precisely:

- **Delhivery Prospectus, 2022:** lists Sandeep Kumar Barasia as Executive Director & Chief Business Officer, and Donald Francis Colleran and Suvir Suren Sujaan as Non-Executive Nominee Directors — all active.
- **Delhivery Annual Report FY24:** the same three are recorded as no longer on the board — Suvir Suren Sujaan resigned effective August 24, 2023; Donald Francis Colleran ceased effective September 27, 2023; Sandeep Kumar Barasia ceased effective July 1, 2024.

This is a genuinely different *kind* of fact than the GDP/revenue examples — predicate: "board position", value: "active" vs "resigned", reconciled by period/as_of rather than by unit or scope. It's non-numeric, it's categorical, and it's the one example the assignment itself uses.

Why this matters for your build, not just your demo: if your extraction prompt and schema are implicitly numbers-shaped (value: float, unit: str), this case will slip through extraction entirely, and you'll have satisfied the letter of "four cases" with three numeric ones and never produced the case the brief actually illustrates. Make sure value is typed to accept a string/categorical result too, and make sure your page-prioritization heuristic (2.1) doesn't only look for numbers.

Confidence: high — direct extraction from both PDFs, cross-checked by name.

2.6 Schema fix this implies

Your scope field needs to carry the *basis/definition*, not just a geography or business-unit label, and value needs to accept categorical facts, not just numbers:

```
value: number | string          # "6.5" or "resigned" are both valid
scope: "standalone" | "consolidated" | "services (ex. traded goods)" | ...
qualifiers: ["first advance estimate", "provisional estimate", ...]
```

Treat scope + qualifiers as first-class matching keys, not free text you only read after a mismatch is already flagged. Two facts with identical subject/predicate/value/period but different scope should route toward **RECONCILABLE**, not **CORROBORATES**.

One refinement worth adding on top of the base schema: keep the comparison itself as a structured object, not just a label:

```
FactComparison:
  fact_a_id
  fact_b_id
  relationship      # CORROBORATES | CONTRADICTS | RECONCILABLE | ...
  confidence
  reason            # short natural-language explanation
  dimensions:      # what matched vs. what differed
    subject: same | different
    predicate: same | similar | different
    value: same | different
    unit: same | different
    period: same | different
    scope: same | different
```

This costs almost nothing to add and pays for itself twice: it's what makes the "reasoning trace" in your UI actually renderable as a small table instead of a paragraph, and it's what you'd point to in the README's "Engineering Decisions" section as evidence the system reasons about *dimensions*, not just vibes. Whether the dimensions block is filled in by a cheap deterministic diff or by the LLM, don't let a deterministic diff *silently* decide the final relationship on its own — see the hierarchical-classifier note in Section 5. Feed it into the LLM's prompt as a strong hint, and let the model make the final call.

3. Priority fixes, ranked by how much they can sink the submission

#	Risk	Why it matters	Fix
1	Extraction/reasoning quality is unvalidated until day 2	If the LLM step is weak, you find out with little runway left	Run one real page through your prompt today, in the first hour , before writing any scaffolding (3.1)
2	511 pages, naive per-page LLM calls	Cost/latency blowout, possible rate limits mid-build	Cheap deterministic candidate-page scoring before any LLM call (2.1) — a method, not a cutoff, so it still runs on an unseen PDF; use a cheap/fast model for extraction, reserve the stronger model for the reasoning/classification step
3	Candidate matching by exact string match on subject/predicate	Misses “GDP growth” vs “real GDP growth” vs “growth”; misses “Delhivery” vs “Delhivery Limited” vs “the Company”	Start with canonical-string normalization + alias resolution only (5). Add embeddings later, only if you actually hit a matching failure the normalizer can’t fix — don’t build it upfront

#	Risk	Why it matters	Fix
4	Forced JSON schema not actually enforced at generation time	Prompt-and-hope extraction silently produces malformed/hallucinated facts under time pressure	Use tool-calling / structured-output mode so the model <i>can't</i> return non-conforming JSON, not just a Pydantic check after the fact
5	Generalization claim untested	Brief says they may test with unseen PDFs; hand-picking your 4 cases from the starter set is fine only if the underlying logic has zero doc-specific rules	Before recording the demo, run the full pipeline on one PDF that is <i>not</i> in the starter set and confirm it doesn't break
6	Deadline timezone assumption	47-hour math only holds if "11:00 AM" is IST	Confirm once, now — cheap insurance

4. Re-sequenced start (today only — your day 2/3 plan is fine as-is)

Your plan goes: scaffold repo → PDF ingestion → fact extraction → normalization → (day 2) reasoning → API → UI. That's a reasonable *build* order but a risky *validation* order — reasoning quality is both the hardest part and the part the assignment says is actually being judged ("the interesting part is how facts are discovered, grounded, compared, and explained"), and it's scheduled last.

Suggested change — same total time, different order for the first ~90 minutes. No repo, no FastAPI, no DB — just a script that calls the model against hand-picked pages and prints structured JSON. Run four micro-tests:

Test	Pages	Expected result
A — basic extraction	One RBI or IMF page	Clean fact with subject/predicate/value/unit/period/evidence/page, nothing invented
B — numeric reconciliation	Economic Survey (6.4%) + RBI or IMF (6.5%) page	Classified RECONCILABLE , reason references estimate vintage, not "rounding" or "error"

Test	Pages	Expected result
C — scope reconciliation	Two Delhivery Annual Report FY24 revenue paragraphs (standalone vs. consolidated)	Classified RECONCILABLE , scope captured as the differing dimension
D — non-numeric extraction	Delhivery Prospectus 2022 director list + Annual Report FY24 director-changes disclosure (2.5)	A categorical fact extracted at all (value: "resigned"), not silently dropped for not being a number

If A-C pass but D fails, you’ve learned your extraction prompt is implicitly numbers-only *before* you’ve built anything around that assumption — much cheaper to find out now. Only after all four pass: scaffold the repo and build outward.

If any test fails, you’ve lost well under two hours, not a day and a half.

5. Smaller suggestions worth doing if time allows

- **Unit tests for the normalizer and matcher** (currency conversions, period-string parsing, scope-mismatch routing). Cheap, and it gives you concrete material for the README’s “Engineering Decisions” section.
- **Entity canonicalization** — a simple alias table built at extraction time (“Delhivery Limited” → “Delhivery”) so cross-document grouping doesn’t silently split the same entity into two subjects. Do this before reaching for embeddings (3).
- **Reasoning trace in the output** (already in your plan) — keep it, it’s the clearest way to show “explained,” not just “classified.” Render it from the dimensions object in 2.6, not a hand-written string.
- **Cheap deterministic checks as hints, not verdicts.** A rule like “different scope → likely RECONCILABLE” is a useful prior to hand the LLM, and cuts cost by skipping the LLM entirely on obviously-identical pairs — but don’t let a hard-coded rule *output* the final relationship for the ambiguous cases. A scope mismatch can also mean your extractor misread something. Keep the LLM as the last word when the deterministic signal is anything short of certain.
- **If you use the “naive extractor fails, then we fixed it” framing for Case 4** (a stronger story than a still-broken demo): keep evidence of the failure, not just a description of it. A saved JSON output, a screenshot, or a small regression test showing the v0 misclassification, committed to the repo before the fix — otherwise “we found a bug and fixed it” is unverifiable and reads the same as “we’re claiming credit for a bug we never actually had.”
- Keep the UI as plain HTML/JS as planned — nothing above changes that call.

6. Open questions to settle before you start the clock

1. Confirm “11:00 AM” deadline timezone (IST assumed, from a VIT Vellore sender).
2. Which LLM/API you’re actually using — cost and rate limits differ enough across providers that this affects the page-sampling decision in 2.1/3.2.
3. Whether you’re comfortable disclosing a page-prioritization method in the README (recommended, see 2.1) vs. trying to process all 511 pages for real (higher risk, not obviously higher reward given the grading criteria).

v1 of this review was based on direct inspection of starter-datasets.zip (both dataset READMEs and text extracted from all 6 PDFs) plus the assignment PDF and the shortlist email. v2 additionally incorporates a second AI's counter-review and one further dataset finding (2.5, the director-status case), found by extracting director names from the 2022 prospectus and the FY24 annual report and cross-checking them. Figures and statements quoted above are paraphrased from the source filings for identification purposes.